MODUL 11 · HIVE NOSQL

Database NoSQL Lokal dengan Hive

Menyimpan data offline dengan Hive Community Edition (hive_ce): box key-value, Map dan List, objek sendiri dengan TypeAdapter manual maupun hasil generate, UI reaktif, Hive dengan GetX + MVC (service dan controller), dan enkripsi. Ditutup dengan mini project CRUD Catatan Kuliah Offline lengkap dengan pencarian dan pengurutan.

Durasi Pertemuan 11 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management dengan GetX + MVC
Modul 10: Penyimpanan Data Lokal (Storage & SQLite)

Capaian Pembelajaran

- Menjelaskan konsep box key-value pada Hive dan memilih antara SharedPreferences, Hive, dan SQLite sesuai kebutuhan.
- Menginisialisasi Hive dan membuka box dengan benar di main() menggunakan hive_ce_flutter.
- Melakukan operasi put, get, add, delete, keys, values, length, dan clear pada box.
- Menyimpan Map dan List serta mengonversinya dengan aman menjadi objek model (toMap/fromMap).
- Menulis TypeAdapter manual dan membuat adapter otomatis dengan @GenerateAdapters, termasuk aturan typeId dan migrasi field.
- Membangun UI reaktif dengan ValueListenableBuilder serta memisahkan service Hive, GetxController, dan view (pola MVC dengan GetX).
- Mengenkripsi box dengan HiveAesCipher dan menyimpan kuncinya di flutter_secure_storage.
- Membangun aplikasi CRUD offline lengkap (tambah, lihat, edit, hapus, cari, urutkan) dan mengujinya dengan unit test.

Daftar Isi Modul

1. Apa itu Hive? Box, NoSQL, dan Kapan Memakainya
2. Instalasi dan Inisialisasi
3. Operasi Dasar Box
4. Menyimpan Map dan List
5. Objek Sendiri dengan TypeAdapter Manual
6. Generate Adapter Otomatis (Lanjutan)
7. UI Reaktif dengan ValueListenableBuilder
8. Hive + GetX Controller (Pola MVC)
9. Enkripsi Box
10. Mini Project: Catatan Kuliah Offline (CRUD Lengkap)

11. Kesalahan Umum dan Debugging
12. Tugas Praktikum
13. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Hive? Box, NoSQL, dan Kapan Memakainya

Di Modul 10 kamu sudah memakai **SharedPreferences** untuk data kecil dan **SQLite** untuk data berbentuk tabel. Modul ini mengenalkan pilihan ketiga yang sangat populer di Flutter: **Hive**, database NoSQL yang ringan, cepat, dan ditulis murni dengan Dart.

Hive itu apa?

Hive adalah database *key-value*. Bayangkan sebuah `Map` di Dart yang isinya otomatis tersimpan ke file, sehingga tidak hilang saat aplikasi ditutup. Kamu menyimpan data dengan sebuah **key** (kunci) lalu mengambilnya lagi memakai key yang sama.

```
// Map biasa: hilang saat aplikasi ditutup
final dataSementara = <String, dynamic>{};
dataSementara['nama'] = 'Budi';

// Box Hive: cara pakainya mirip Map, tetapi tersimpan permanen ke file
// box.put('nama', 'Budi');
// box.get('nama'); // 'Budi', bahkan setelah aplikasi dibuka ulang
```

Istilah penting: Box

Data di Hive dikelompokkan ke dalam **box**. Satu box kira-kira setara dengan satu tabel di SQLite atau satu "laci" penyimpanan. Contoh pembagiannya:

- `pengaturan`: tema gelap, bahasa, nama pengguna.
- `catatan_box`: semua catatan kuliah.
- `cache_mahasiswa`: data dari server yang disimpan sementara agar bisa dibuka saat offline.

Setiap box disimpan sebagai file sendiri (`nama_box.hive`) di folder dokumen aplikasi. Box yang sudah dibuka isinya dimuat ke memori, sehingga membaca data dari box **tidak perlu** `await` dan hasilnya langsung didapat.

NoSQL: tidak ada tabel dan kolom

Berbeda dengan SQLite, Hive tidak memakai `CREATE TABLE`, tidak ada skema kolom, dan tidak ada query SQL. Value yang disimpan bisa berupa:

- Tipe dasar: `String`, `int`, `double`, `bool`, `DateTime`, `null`.
- Koleksi: `List` dan `Map` (boleh bersarang).
- Objek buatan sendiri (misalnya `Catatan`), asalkan kamu mendaftarkan **TypeAdapter** (dibahas di bagian 5).

Kapan memakai Hive, SharedPreferences, atau SQLite?

Aspek	SharedPreferences	Hive	SQLite (sqflite)
Model data	Key-value sederhana	Key-value NoSQL, value boleh objek	Relasional (tabel, baris, kolom)
Tipe value	String, int, double, bool, List<String>	Semua tipe dasar, List, Map, objek dengan adapter	TEXT, INTEGER, REAL, BLOB
Cara baca	`await` (async)	Sinkron setelah box dibuka	`await` + query SQL
Query rumit (JOIN, GROUP BY)	Tidak bisa	Tidak ada; filter manual dengan `where()` di Dart	Sangat bisa
Ditulis dengan	Plugin native	Murni Dart (jalan juga di web dan desktop)	Plugin native
Cocok untuk	Flag kecil: tema, sudah onboarding, token	Daftar objek, cache offline, catatan, keranjang belanja	Data besar yang saling berelasi, laporan, agregasi

> 💡 **Aturan praktis**
>
> Cuma beberapa pengaturan? Pakai SharedPreferences. Butuh menyimpan daftar objek dengan cepat dan tanpa SQL? Pakai Hive. Butuh relasi antar tabel dan query kompleks? Pakai SQLite. Ketiganya boleh dipakai bersamaan dalam satu aplikasi.

Kenapa `hive_ce`, bukan `hive`?

Di pub.dev ada package `hive` versi 2.x yang dulu sangat populer. Sayangnya package itu sudah lama **tidak dirawat** lagi oleh pembuatnya. Komunitas lalu melanjutkannya sebagai **Hive Community Edition**: `hive_ce` dan `hive_ce_flutter`. API-nya hampir sama persis (`Hive.openBox`, `box.put`, `TypeAdapter`, dan seterusnya), tetapi rutin diperbarui untuk versi Dart dan Flutter terbaru, serta ada tambahan seperti `@GenerateAdapters` dan rentang `typeId` yang jauh lebih besar. Jadi tutorial lama tentang `hive` umumnya tetap bisa diikuti; yang berbeda hanya nama package dan baris `import`.

ℹ️ **Peta modul ini**

Kita mulai dari operasi dasar box, menyimpan Map dan List, lalu objek sendiri dengan TypeAdapter. Setelah itu UI reaktif, menggabungkan Hive dengan GetX + MVC (service dan controller), enkripsi, dan diakhiri mini project **Catatan Kuliah Offline** (CRUD lengkap). Teknik menyimpan Map di bagian 4 akan dipakai lagi di Modul 14 sebagai cache offline data Supabase.

2. Instalasi dan Inisialisasi

Sebelum bisa menyimpan data, Hive perlu dipasang, diberi tahu folder penyimpanannya, lalu box yang dibutuhkan dibuka. Semua itu dilakukan sekali di `main()`.

Langkah 1: Pasang package

Buat proyek baru (atau pakai proyek yang ada), lalu jalankan dari folder proyek:

```
flutter create halo_hive
cd halo_hive
flutter pub add hive_ce hive_ce_flutter
```

- `hive_ce`: inti database (murni Dart).
- `hive_ce_flutter`: tambahan khusus Flutter, misalnya `Hive.initFlutter()` dan `box.listenable()`.

Hasilnya di `pubspec.yaml` kurang lebih seperti ini (nomor versi mengikuti yang terbaru saat kamu menjalankan perintah):

```yaml
dependencies:
  flutter:
    sdk: flutter
  hive_ce: ^x.y.z
  hive_ce_flutter: ^x.y.z
```

Langkah 2: Import

Di file Flutter cukup satu baris ini, karena `hive_ce_flutter` sudah ikut meng-export seluruh isi `hive_ce`:

```dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
```

Untuk file Dart murni yang tidak menyentuh Flutter (misalnya model), boleh juga memakai `import 'package:hive_ce/hive_ce.dart';`.

Langkah 3: Inisialisasi di `main()`

Urutannya selalu sama:

1. `WidgetsFlutterBinding.ensureInitialized()`: wajib karena kita memanggil kode async yang memakai plugin (mencari folder dokumen) **sebelum** `runApp`.
2. `await Hive.initFlutter()`: menentukan folder penyimpanan.
3. `Hive.registerAdapter(...)`: jika memakai objek sendiri (bagian 5).
4. `await Hive.openBox(...)`: membuka box yang akan dipakai.
5. `runApp(...)`.

Contoh lengkap: aplikasi yang menghitung sudah berapa kali ia dibuka. Hapus seluruh isi `lib/main.dart` lalu ganti dengan kode berikut.

lib/main.dart

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> main() async {
  // Wajib dipanggil sebelum await apa pun yang memakai plugin
  WidgetsFlutterBinding.ensureInitialized();

  // Menentukan folder penyimpanan Hive (folder dokumen aplikasi)
  await Hive.initFlutter();

  // Membuka box bernama 'pengaturan'. Jika belum ada, Hive membuatnya.
  final box = await Hive.openBox('pengaturan');

  // Baca hitungan lama (default 0), tambah 1, simpan lagi
  final jumlahBuka = (box.get('jumlah_buka', defaultValue: 0) as int) + 1;
  await box.put('jumlah_buka', jumlahBuka);

  runApp(MyApp(jumlahBuka: jumlahBuka));
}

class MyApp extends StatelessWidget {
  final int jumlahBuka;

  const MyApp({super.key, required this.jumlahBuka});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Halo Hive')),
        body: Center(
          child: Text(
            'Aplikasi ini sudah dibuka $jumlahBuka kali',
            style: const TextStyle(fontSize: 20),
          ),
        ),
      ),
    );
  }
}
```



Angka bertambah setiap aplikasi dijalankan ulang, karena tersimpan di box

Penjelasan:

- `Future<void> main() async` : `main` boleh async agar bisa menunggu Hive siap sebelum UI tampil.
- `Hive.openBox('pengaturan')` membuat file `pengaturan.hive` jika belum ada. Box yang tidak diberi tipe bertipe `Box<dynamic>`, jadi boleh diisi tipe apa saja.
- `box.get('jumlah_buka', defaultValue: 0)` mengembalikan 0 saat aplikasi pertama kali dibuka.
- Coba tutup aplikasi sepenuhnya lalu buka lagi: angkanya bertambah. Itulah bukti data tersimpan permanen.

Mengambil box yang sudah dibuka

Membuka box itu operasi async (membaca file). Setelah terbuka, di mana pun di aplikasi kamu cukup memanggil `Hive.box()` yang sinkron:

```dart
// Di main(): sekali saja, pakai await
await Hive.openBox('pengaturan');

// Di widget atau kelas lain: langsung ambil, tanpa await
final box = Hive.box('pengaturan');
final tema = box.get('tema_gelap', defaultValue: false);

// Box bertipe: semua value wajib bertipe String
await Hive.openBox<String>('tugas_cepat');
final tugasBox = Hive.box<String>('tugas_cepat');
```

⚠️ **Kesalahan umum**

- Lupa `WidgetsFlutterBinding.ensureInitialized()` menyebabkan error "Binding has not yet been initialized".
- Memanggil `Hive.box('x')` padahal `openBox('x')` belum dipanggil memicu `HiveError: Box not found. Did you forget to call Hive.openBox()?`
- Membuka box yang sama dengan tipe berbeda (`openBox<String>('a')` lalu `openBox('a')`) juga memicu error. Pakai tipe yang sama secara konsisten.

3. Operasi Dasar Box

Box punya method yang mirip `Map`. Di bagian ini kita coba satu per satu, lengkap dengan output yang muncul di Debug Console.

Daftar method penting

Method	Fungsi	Async?
`put(key, value)`	Simpan atau timpa value dengan key tertentu	Ya (`Future`)
`get(key, defaultValue: ...)`	Ambil value; `null` atau `defaultValue` jika key tidak ada	Tidak
`add(value)`	Simpan dengan key angka otomatis (0, 1, 2, ...), mengembalikan key baru	Ya
`delete(key)`	Hapus satu data berdasarkan key	Ya
`containsKey(key)`	Cek apakah key ada	Tidak
`keys`, `values`, `length`	Semua key, semua value, jumlah data	Tidak
`getAt(i)`, `putAt(i, v)`, `deleteAt(i)`	Akses berdasarkan posisi (index), bukan key	`getAt` tidak, lainnya ya
`toMap()`	Salin seluruh isi box menjadi `Map`	Tidak
`clear()`	Hapus semua data di box, mengembalikan jumlah yang terhapus	Ya

Pola mudahnya: **menulis** (put, add, delete, clear) mengembalikan `Future`; **membaca** (get, keys, values, length) langsung dapat hasilnya.

Contoh lengkap semua operasi

Fungsi berikut bisa kamu panggil dari `main()` setelah `await Hive.initFlutter()`, misalnya `await demoOperasiDasar();` sebelum `runApp`.

```
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> demoOperasiDasar() async {
  final box = await Hive.openBox('latihan');
```

```dart
  // ---- CREATE / UPDATE dengan key buatan sendiri ----
  await box.put('nama', 'Budi');
  await box.put('semester', 4);
  await box.put('aktif', true);
  await box.put('semester', 5); // key sama = menimpa (update)

  // ---- READ ----
  debugPrint('${box.get('nama')}'); // Budi
  debugPrint('${box.get('semester')}'); // 5
  debugPrint('${box.get('ipk')}'); // null (key belum ada)
  debugPrint('${box.get('ipk', defaultValue: 0.0)}'); // 0.0
  debugPrint('${box.containsKey('aktif')}'); // true

  // ---- add(): key berupa angka otomatis 0, 1, 2, ... ----
  final k1 = await box.add('Kalkulus');
  final k2 = await box.add('Basis Data');
  debugPrint('key baru: $k1, $k2');

  // ---- melihat isi box ----
  debugPrint('keys   : ${box.keys}');
  debugPrint('values : ${box.values}');
  debugPrint('length : ${box.length}');

  // ---- akses berdasarkan POSISI (index), bukan key ----
  debugPrint('getAt(0): ${box.getAt(0)}');
  await box.putAt(0, 'Kalkulus Lanjut');
  debugPrint('setelah putAt: ${box.get(0)}');

  // ---- DELETE ----
  await box.delete('aktif');
  await box.deleteAt(0);
  debugPrint('setelah delete: ${box.toMap()}');

  final jumlahDihapus = await box.clear();
  debugPrint('clear() menghapus $jumlahDihapus data, sisa ${box.length}');
}
```

Output di Debug Console:

```
Budi
5
null
0.0
true
key baru: 0, 1
keys   : (0, 1, aktif, nama, semester)
values : (Kalkulus, Basis Data, true, Budi, 5)
length : 5
getAt(0): Kalkulus
setelah putAt: Kalkulus Lanjut
setelah delete: {1: Basis Data, nama: Budi, semester: 5}
clear() menghapus 3 data, sisa 0
```

Hal yang perlu diperhatikan dari output:

- `put` **= create sekaligus update.** `put('semester', 5)` menimpa nilai 4 karena key-nya sama. Hive tidak membedakan insert dan update.
- **Key boleh** `String` **atau** `int`**.** `add()` memakai key int otomatis yang terus naik.
- **Urutan key bukan urutan penyimpanan.** Hive mengurutkan key: angka dulu, lalu string secara alfabet. Karena itu `getAt(0)` menghasilkan `Kalkulus` (key 0), bukan `Budi` yang disimpan pertama.
- `deleteAt(0)` **menghapus posisi pertama**, yaitu key 0 (`Kalkulus Lanjut`), sedangkan `delete('aktif')` menghapus berdasarkan key.

> 💡 **Pakai key, hindari index**
>
> Method `...At(index)` mudah salah, karena index bergeser setiap ada data yang dihapus. Untuk data seperti catatan atau tugas, simpan dengan key unik milik datanya sendiri (misalnya `id`) memakai `put(id, data)`, lalu ubah dan hapus memakai key yang sama. Pola ini dipakai di mini project.

Perlu `await` saat menulis?

Data yang di-`put` langsung bisa dibaca dari memori meskipun tidak di-`await`. Namun penulisan ke file berjalan di belakang layar. Jika aplikasi tiba-tiba dimatikan sebelum `Future`-nya selesai, data itu bisa hilang. Biasakan `await` di method async, terutama sebelum pindah halaman atau menampilkan pesan "tersimpan".

```
// Kurang aman: tidak menunggu sampai benar-benar tertulis ke file
box.put('draft', teks);

// Aman
await box.put('draft', teks);
```

4. Menyimpan Map dan List

Tanpa membuat adapter apa pun, Hive sudah bisa menyimpan `List` dan `Map`. Ini cara tercepat menyimpan data terstruktur, dan juga cara yang dipakai untuk cache data JSON dari API atau Supabase.

Menyimpan dan membaca List & Map

```dart
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> demoMapList() async {
  final box = await Hive.openBox('profil_box');

  // Menyimpan List dan Map langsung, tanpa adapter
  await box.put('matkul_favorit', ['Pemrograman Mobile', 'Basis Data', 'AI']);
  await box.put('profil', {
    'nama': 'Siti Aminah',
    'nim': 'H1D023001',
    'ipk': 3.75,
    'hobi': ['membaca', 'coding'],
  });

  final favorit = box.get('matkul_favorit') as List;
  debugPrint('Jumlah favorit: ${favorit.length}');
  debugPrint('Pertama: ${favorit.first}');

  final profil = box.get('profil') as Map;
  debugPrint('Nama: ${profil['nama']}, IPK: ${profil['ipk']}');
}

Future<void> demoMapSetelahDibukaUlang() async {
  final box = await Hive.openBox('profil_box');
  final mentah = box.get('profil');
  debugPrint('Tipe asli: ${mentah.runtimeType}');

  // final salah = mentah as Map<String, dynamic>; // TypeError!
  final profil = Map<String, dynamic>.from(mentah as Map);
  final hobi = List<String>.from(profil['hobi'] as List);
  debugPrint('${profil['nama']} punya ${hobi.length} hobi');
}
```

Output saat `demoMapList()` dijalankan, lalu aplikasi dibuka ulang dan `demoMapSetelahDibukaUlang()` dijalankan:

```
Jumlah favorit: 3
Pertama: Pemrograman Mobile
Nama: Siti Aminah, IPK: 3.75
Tipe asli: _Map<dynamic, dynamic>
Siti Aminah punya 2 hobi
```

⚠️ Jebakan paling sering: Map<dynamic, dynamic>

Saat disimpan, Map kamu bertipe `Map<String, Object>`. Namun setelah aplikasi dibuka ulang, Hive membacanya kembali dari file sebagai `Map<dynamic, dynamic>`. Akibatnya cast langsung akan gagal:

```
type '_Map<dynamic, dynamic>' is not a subtype of type 'Map<String, dynamic>' in type cast
```

Yang lebih menjebak, error ini **tidak muncul** saat kamu baru saja menyimpan data (data masih berupa objek asli di memori), tetapi baru muncul setelah aplikasi di-restart. Solusinya: selalu salin dengan `Map<String, dynamic>.from(...)` dan `List<String>.from(...)`, atau terima parameter bertipe `Map` saja.

Pola model + toMap/fromMap

Agar tidak bekerja dengan Map mentah di seluruh aplikasi, buat kelas model yang bisa diubah ke Map dan sebaliknya. Pola ini sama dengan `toJson/fromJson` di Modul 09.

lib/models/tugas.dart

```dart
// lib/models/tugas.dart
class Tugas {
  final String id;
  final String judul;
  final DateTime deadline;
  final bool selesai;

  const Tugas({
    required this.id,
    required this.judul,
    required this.deadline,
    this.selesai = false,
  });

  // Objek -> Map (untuk disimpan ke Hive, atau dikirim ke API)
  Map<String, dynamic> toMap() => {
        'id': id,
        'judul': judul,
        'deadline': deadline.toIso8601String(),
        'selesai': selesai,
      };

  // Map -> Objek. Parameter bertipe Map (bukan Map<String, dynamic>)
  // karena Map dari Hive bertipe Map<dynamic, dynamic>.
  factory Tugas.fromMap(Map map) => Tugas(
        id: map['id'] as String,
        judul: map['judul'] as String,
        deadline: DateTime.parse(map['deadline'] as String),
        selesai: map['selesai'] as bool? ?? false,
```

```
      );
}
```

lib/repositories/tugas_repository.dart

```dart
// lib/repositories/tugas_repository.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import '../models/tugas.dart';

class TugasRepository {
  // Box<Map>: setiap value adalah Map
  final Box<Map> _box = Hive.box<Map>('tugas_box');

  Future<void> simpan(Tugas tugas) => _box.put(tugas.id, tugas.toMap());

  List<Tugas> ambilSemua() =>
      _box.values.map((map) => Tugas.fromMap(map)).toList();

  Future<void> hapus(String id) => _box.delete(id);
}
```

Jangan lupa membuka box-nya dengan tipe yang sama di `main()`: `await Hive.openBox<Map>('tugas_box');`

Kenapa `DateTime` diubah menjadi String ISO? Sebenarnya Hive bisa menyimpan `DateTime` langsung di dalam Map. Namun memakai `toIso8601String()` membuat Map yang sama juga bisa dikirim sebagai JSON ke server tanpa perubahan. Itu berguna ketika data lokal harus disinkronkan dengan backend.

> 💡 **Map atau TypeAdapter?**
>
> - **Map (bagian ini):** tanpa adapter, tidak ada `typeId`, mudah ditambah field baru (field lama yang tidak ada cukup diberi nilai default di `fromMap`). Cocok untuk cache data dari API dan untuk data yang strukturnya masih sering berubah.
> - **TypeAdapter (bagian 5 dan 6):** objek tersimpan langsung, `box.get()` sudah mengembalikan objek bertipe tepat, dan file lebih ringkas. Cocok untuk model yang stabil.

5. Objek Sendiri dengan TypeAdapter Manual

Jika kamu mencoba `box.put('a', Mahasiswa(...))` tanpa persiapan, Hive akan melempar `HiveError: Cannot write, unknown type: Mahasiswa. Did you forget to register an adapter?`. Hive tidak tahu cara mengubah objek buatanmu menjadi byte. Tugas itu dikerjakan oleh **TypeAdapter**.

Cara kerja TypeAdapter

TypeAdapter adalah kelas yang berisi tiga hal:

1. `typeId` : nomor unik untuk tipe ini. Hive menulis nomor ini di depan data agar tahu adapter mana yang harus dipakai saat membaca.
2. `write()` : mengubah objek menjadi urutan nilai (serialisasi).
3. `read()` : membaca urutan nilai itu kembali menjadi objek (deserialisasi).

Kita menulis adapter secara manual dulu (tanpa code generator) supaya jelas apa yang sebenarnya terjadi.

Langkah 1: Model dan adapter

lib/models/mahasiswa.dart

```dart
// lib/models/mahasiswa.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class Mahasiswa {
  final String nim;
  final String nama;
  final int semester;
  final double ipk;

  const Mahasiswa({
    required this.nim,
    required this.nama,
    required this.semester,
    required this.ipk,
  });

  @override
  String toString() => 'Mahasiswa($nim, $nama, smt $semester, IPK $ipk)';
}

class MahasiswaAdapter extends TypeAdapter<Mahasiswa> {
  @override
  final int typeId = 1; // nomor unik untuk kelas Mahasiswa

  @override
  Mahasiswa read(BinaryReader reader) {
    // Urutan baca = urutan tulis
    final nim = reader.readString();
    final nama = reader.readString();
    final semester = reader.readInt();
```

```
    final ipk = reader.readDouble();
    return Mahasiswa(nim: nim, nama: nama, semester: semester, ipk: ipk);
  }

  @override
  void write(BinaryWriter writer, Mahasiswa obj) {
    writer.writeString(obj.nim);
    writer.writeString(obj.nama);
    writer.writeInt(obj.semester);
    writer.writeDouble(obj.ipk);
  }
}
```

Method yang tersedia di `BinaryWriter` dan `BinaryReader` berpasangan: `writeString/readString`, `writeInt/readInt`, `writeDouble/readDouble`, `writeBool/readBool`, `writeByte/readByte`, `writeList/readList`, `writeMap/readMap`, serta `write/read` yang otomatis memakai tipe yang dikenal Hive.

Langkah 2: Daftarkan adapter lalu pakai box bertipe

```
import 'package:flutter/foundation.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'models/mahasiswa.dart';

Future<void> demoMahasiswa() async {
  // Di main(): setelah Hive.initFlutter(), sebelum openBox
  Hive.registerAdapter(MahasiswaAdapter());

  final box = await Hive.openBox<Mahasiswa>('mahasiswa_box');

  await box.put(
    'H1D023001',
    const Mahasiswa(nim: 'H1D023001', nama: 'Siti', semester: 4, ipk: 3.75),
  );
  await box.put(
    'H1D023002',
    const Mahasiswa(nim: 'H1D023002', nama: 'Andi', semester: 4, ipk: 3.20),
  );

  final siti = box.get('H1D023001'); // tipe: Mahasiswa?
  debugPrint('$siti');
  debugPrint('IPK Siti: ${siti?.ipk}');

  final ipkTinggi = box.values.where((m) => m.ipk >= 3.5).toList();
  debugPrint('IPK >= 3.5: ${ipkTinggi.map((m) => m.nama).toList()}');
}
```

Output:

```
Mahasiswa(H1D023001, Siti, smt 4, IPK 3.75)
IPK Siti: 3.75
IPK >= 3.5: [Siti]
```

Perhatikan bahwa `box.get()` langsung mengembalikan `Mahasiswa?`, bukan Map. Filter seperti "IPK minimal 3.5" dikerjakan dengan `where()` biasa di Dart.

Aturan typeId

- **Unik per aplikasi.** Dua adapter dengan `typeId` sama memicu `HiveError: There is already a TypeAdapter for typeId 1`.
- **Jangan pernah diganti** setelah aplikasi dipakai orang. Data lama di file tetap bertanda typeId lama, sehingga tidak bisa dibaca lagi.
- Rentang yang diizinkan `hive_ce` adalah 0 sampai 65439 (package `hive` lama hanya 0 sampai 223). Mulailah dari 0, 1, 2, dan seterusnya.
- `Hive.initFlutter()` sudah mendaftarkan adapter bawaan untuk `Color` (typeId 200) dan `TimeOfDay` (typeId 201). Hindari dua nomor itu.
- Daftarkan adapter **sebelum** `openBox` yang menyimpan tipe tersebut.

Aturan urutan field

Adapter manual membaca data secara berurutan seperti kaset: tidak ada nama field di file, hanya nilai-nilai yang berjajar. Jika di `write()` urutannya nim, nama, semester, ipk, maka `read()` wajib membaca nim, nama, semester, ipk juga. Tertukar sedikit saja (misalnya `readDouble` sebelum `readInt`) akan menghasilkan data aneh atau error `RangeError`.

Hati-hati saat model berubah (migrasi)

Bayangkan aplikasi sudah dipakai, lalu kamu menambah field `email`. Data lama di HP pengguna tidak punya email. Jika `read()` langsung memanggil `readString()` keempat, Hive membaca melewati batas data dan aplikasi crash.

Trik sederhana: tulis **nomor versi format** sebagai byte pertama sejak awal. Saat membaca, field baru hanya dibaca jika versinya mendukung.

```
// Versi 2 dari model: field baru `email`
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class Dosen {
  final String nidn;
  final String nama;
  final String email; // FIELD BARU di versi 2

  const Dosen({required this.nidn, required this.nama, this.email = ''});
}

class DosenAdapter extends TypeAdapter<Dosen> {
  @override
  final int typeId = 2;

  static const int _versiSekarang = 2;

  @override
  Dosen read(BinaryReader reader) {
```

```
    final versi = reader.readByte(); // byte pertama = versi format
    final nidn = reader.readString();
    final nama = reader.readString();
    // Data versi 1 belum punya email, jadi hanya dibaca jika versi >= 2
    final email = versi >= 2 ? reader.readString() : '';
    return Dosen(nidn: nidn, nama: nama, email: email);
  }

  @override
  void write(BinaryWriter writer, Dosen obj) {
    writer.writeByte(_versiSekarang);
    writer.writeString(obj.nidn);
    writer.writeString(obj.nama);
    writer.writeString(obj.email); // field baru selalu DITAMBAH DI AKHIR
  }
}
```

⚠️ **Aturan migrasi adapter manual**

- Field baru selalu **ditambah di akhir**, jangan disisipkan di tengah.
- Jangan menghapus atau mengubah tipe field lama. Jika tidak dipakai lagi, tetap baca lalu abaikan nilainya.
- Selama masih tahap belajar dan data boleh hilang, cara paling cepat adalah menghapus aplikasi dari emulator (uninstall) agar file box ikut terhapus.
- Kode yang dibuat generator (bagian 6) menulis nomor indeks untuk setiap field, sehingga migrasi jauh lebih aman. Itu alasan utama memakai codegen di proyek nyata.

6. Generate Adapter Otomatis (Lanjutan)

Menulis adapter manual untuk banyak model cukup melelahkan dan rawan salah urutan. `hive_ce_generator` bisa membuatkan adapter secara otomatis. Bagian ini bersifat **lanjutan (opsional)**; pastikan kamu sudah paham bagian 5 dulu.

Langkah 1: Pasang generator

```
flutter create rak_buku
cd rak_buku
flutter pub add hive_ce hive_ce_flutter
flutter pub add dev:hive_ce_generator dev:build_runner
```

Awalan `dev:` menaruh package di `dev_dependencies`, karena generator hanya dipakai saat development dan tidak ikut ke APK.

Langkah 2: Model biasa, tanpa anotasi

`lib/models/buku.dart`

```dart
// lib/models/buku.dart  (class biasa, tanpa anotasi apa pun)
class Buku {
  final String isbn;
  final String judul;
  final int tahun;
  final List<String> penulis;

  Buku({
    required this.isbn,
    required this.judul,
    required this.tahun,
    required this.penulis,
  });
}
```

Langkah 3: Daftarkan model di satu file `@GenerateAdapters`

Cara yang direkomendasikan `hive_ce` saat ini adalah anotasi `@GenerateAdapters`. Semua model yang ingin dibuatkan adapter cukup didaftarkan di satu tempat; model tidak perlu ditempeli `@HiveType` dan `@HiveField` seperti di tutorial `hive` lama.

`lib/hive/hive_adapters.dart`

```dart
// lib/hive/hive_adapters.dart
import 'package:hive_ce/hive_ce.dart';

import '../models/buku.dart';

part 'hive_adapters.g.dart';
```

```
@GenerateAdapters([AdapterSpec<Buku>()])
class HiveAdapters {}
```

Langkah 4: Jalankan build_runner

```
dart run build_runner build --delete-conflicting-outputs
```

Tiga file baru akan muncul di folder `lib/hive/`:

File	Isi
`hive_adapters.g.dart`	Kelas `BukuAdapter` hasil generate
`hive_adapters.g.yaml`	Catatan typeId dan indeks tiap field. **Wajib ikut di-commit** ke Git, karena dipakai agar indeks tidak berubah saat model diubah.
`hive_registrar.g.dart`	Extension `Hive.registerAdapters()` untuk mendaftarkan semua adapter sekaligus

Isi `hive_adapters.g.yaml` hasil generate:

```
# Generated by Hive CE
# Manual modifications may be necessary for certain migrations
# Check in to version control
nextTypeId: 1
types:
  Buku:
    typeId: 0
    nextIndex: 4
    fields:
      isbn:
        index: 0
      judul:
        index: 1
      tahun:
        index: 2
      penulis:
        index: 3
```

Dan potongan `BukuAdapter` yang dibuat otomatis. Perhatikan bahwa setiap nilai diawali nomor indeks field (`writeByte(0)`, `writeByte(1)`, ...). Karena itu field baru bisa ditambahkan tanpa merusak data lama.

```
// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'hive_adapters.dart';

// **************************************************************************
// AdaptersGenerator
```

```dart
// **************************************************************************

class BukuAdapter extends TypeAdapter<Buku> {
  @override
  final typeId = 0;

  @override
  Buku read(BinaryReader reader) {
    final numOfFields = reader.readByte();
    final fields = <int, dynamic>{
      for (int i = 0; i < numOfFields; i++) reader.readByte(): reader.read(),
    };
    return Buku(
      isbn: fields[0] as String,
      judul: fields[1] as String,
      tahun: (fields[2] as num).toInt(),
      penulis: (fields[3] as List).cast<String>(),
    );
  }

  @override
  void write(BinaryWriter writer, Buku obj) {
    writer
      ..writeByte(4)
      ..writeByte(0)
      ..write(obj.isbn)
      ..writeByte(1)
      ..write(obj.judul)
      ..writeByte(2)
      ..write(obj.tahun)
      ..writeByte(3)
      ..write(obj.penulis);
  }

  @override
  int get hashCode => typeId.hashCode;

  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is BukuAdapter &&
          runtimeType == other.runtimeType &&
          typeId == other.typeId;
}
```

`lib/hive/hive_registrar.g.dart` **(dibuat otomatis, jangan diedit)**

```dart
// Generated by Hive CE
// Do not modify
// Check in to version control

import 'package:hive_ce/hive_ce.dart';
import 'package:rak_buku/hive/hive_adapters.dart';

extension HiveRegistrar on HiveInterface {
  void registerAdapters() {
    registerAdapter(BukuAdapter());
  }
```

```
}

extension IsolatedHiveRegistrar on IsolatedHiveInterface {
  void registerAdapters() {
    registerAdapter(BukuAdapter());
  }
}
```

Langkah 5: Pakai di `main.dart`

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'models/buku.dart';
import 'hive/hive_registrar.g.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();

  // Satu baris mendaftarkan SEMUA adapter hasil generate
  Hive.registerAdapters();

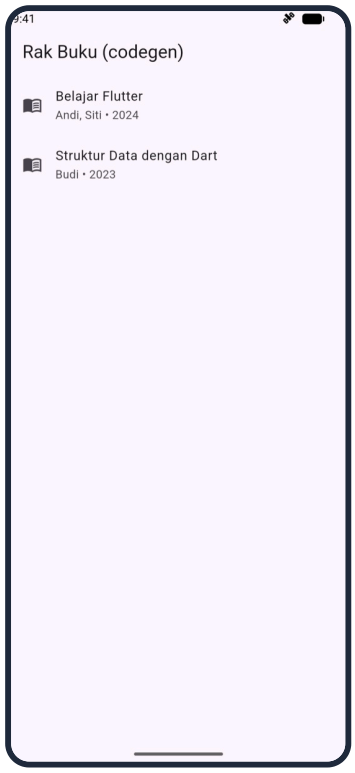
  final box = await Hive.openBox<Buku>('buku_box');
  if (box.isEmpty) {
    await box.add(Buku(
      isbn: '978-602-1',
      judul: 'Belajar Flutter',
      tahun: 2024,
      penulis: ['Andi', 'Siti'],
    ));
    await box.add(Buku(
      isbn: '978-602-2',
      judul: 'Struktur Data dengan Dart',
      tahun: 2023,
      penulis: ['Budi'],
    ));
  }

  runApp(const BukuApp());
}

class BukuApp extends StatelessWidget {
  const BukuApp({super.key});

  @override
  Widget build(BuildContext context) {
    final daftar = Hive.box<Buku>('buku_box').values.toList();
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Rak Buku (codegen)')),
        body: ListView(
          children: [
            for (final b in daftar)
```

```
              ListTile(
                leading: const Icon(Icons.menu_book),
                title: Text(b.judul),
                subtitle: Text('${b.penulis.join(', ')} • ${b.tahun}'),
              ),
            ],
          ),
        ),
      );
    }
  }
```



Objek Buku (termasuk List penulis) tersimpan memakai adapter hasil generate

Saat model berubah

- Tambah field: tambahkan di kelas, jalankan lagi `build_runner`. Field baru mendapat indeks berikutnya (lihat `nextIndex` di file yaml). Untuk data lama, field baru bernilai `null`, jadi buat field baru *nullable* atau beri nilai default di constructor.
- Hapus field: hapus dari kelas lalu generate ulang. Indeksnya tidak akan dipakai ulang.
- Tambah model baru: tambahkan `AdapterSpec<ModelBaru>()` ke daftar `@GenerateAdapters`.

ℹ️ **Bagaimana dengan @HiveType dan @HiveField?**

Gaya lama (anotasi `@HiveType(typeId: 0)` di kelas dan `@HiveField(0)` di setiap field, ditambah `part 'buku.g.dart';`) masih didukung `hive_ce_generator` agar proyek lama tetap jalan. Untuk proyek baru, gunakan `@GenerateAdapters` karena model tetap bersih dan indeks dikelola otomatis lewat file yaml.

7. UI Reaktif dengan ValueListenableBuilder

Kalau data di box berubah, bagaimana UI ikut berubah? Kamu bisa saja memanggil `setState` setelah setiap `put`, tetapi Hive menyediakan cara yang lebih rapi: `box.listenable()` yang dipasangkan dengan widget `ValueListenableBuilder`.

Konsep

- `box.listenable()` (dari `hive_ce_flutter`) menghasilkan `ValueListenable<Box>` yang memberi kabar setiap ada `put`, `add`, `delete`, atau `clear`.
- `ValueListenableBuilder` menjalankan ulang fungsi `builder`-nya setiap kabar itu datang.
- Hasilnya: cukup tulis ke box, dan UI akan menyesuaikan sendiri. Tidak perlu `setState`.

```
ValueListenableBuilder<Box<String>>(
  valueListenable: Hive.box<String>('tugas_cepat').listenable(),
  builder: (context, box, child) {
    return Text('Jumlah tugas: ${box.length}');
  },
)
```

Contoh lengkap: daftar Tugas Cepat

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await Hive.openBox<String>('tugas_cepat');
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const TugasScreen(),
    );
  }
}

class TugasScreen extends StatefulWidget {
  const TugasScreen({super.key});

  @override
  State<TugasScreen> createState() => _TugasScreenState();
```

```dart
}

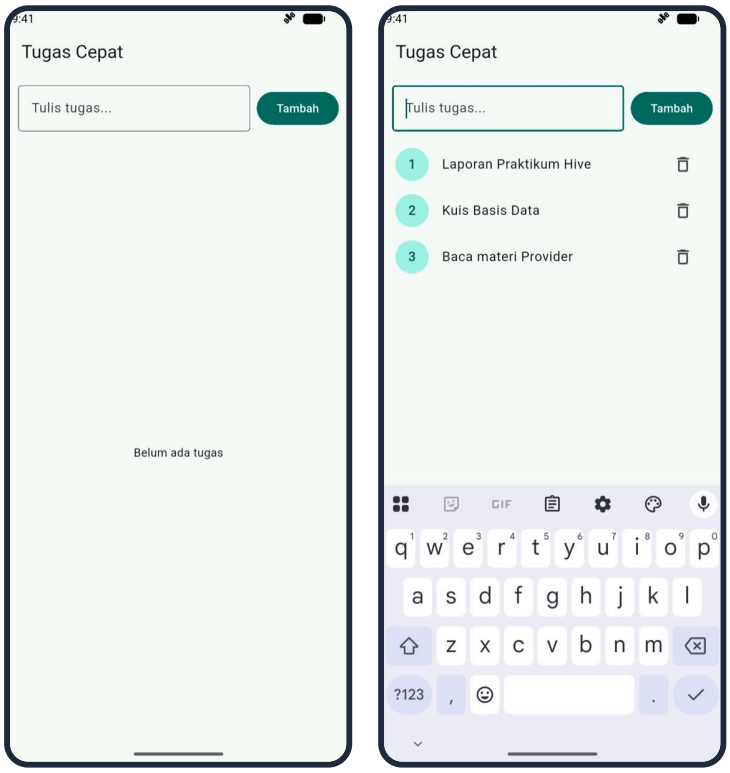
class _TugasScreenState extends State<TugasScreen> {
  // Box sudah dibuka di main(), di sini cukup diambil
  final Box<String> _box = Hive.box<String>('tugas_cepat');
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _tambah() {
    final teks = _controller.text.trim();
    if (teks.isEmpty) return;
    _box.add(teks); // TIDAK perlu setState: UI ikut berubah sendiri
    _controller.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tugas Cepat')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _controller,
                    decoration: const InputDecoration(
                      hintText: 'Tulis tugas...',
                      border: OutlineInputBorder(),
                    ),
                    onSubmitted: (_) => _tambah(),
                  ),
                ),
                const SizedBox(width: 8),
                FilledButton(onPressed: _tambah, child: const Text('Tambah')),
              ],
            ),
          ),
          Expanded(
            // Builder ini dijalankan ulang setiap isi box berubah
            child: ValueListenableBuilder<Box<String>>(
              valueListenable: _box.listenable(),
              builder: (context, box, _) {
                if (box.isEmpty) {
                  return const Center(child: Text('Belum ada tugas'));
                }
                final keys = box.keys.toList();
                return ListView.builder(
                  itemCount: keys.length,
                  itemBuilder: (context, index) {
                    final key = keys[index];
                    return ListTile(
```

```
                    leading: CircleAvatar(child: Text('${index + 1}')),
                    title: Text(box.get(key) ?? ''),
                    trailing: IconButton(
                      icon: const Icon(Icons.delete_outline),
                      onPressed: () => box.delete(key),
                    ),
                  );
                },
              );
            },
          ),
        ),
      ],
    ),
  );
 }
}
```



Box masih kosong

Setelah menambah tugas: list langsung diperbarui tanpa setState

Penjelasan:

- `_box.add(teks)` di `_tambah()` tidak diikuti `setState`. Builder tetap dijalankan ulang karena mendengarkan box.
- Di dalam builder kita memakai `box.keys.toList()`, lalu menghapus berdasarkan **key** (`box.delete(key)`), bukan index. Ingat bagian 3: key dari `add()` terus naik dan tidak ikut bergeser saat ada yang dihapus.
- `setState` masih wajar dipakai untuk state UI lain yang bukan isi box, misalnya teks di `TextField`.

Hanya mendengarkan key tertentu

Jika widget hanya peduli pada satu atau dua key, berikan parameter `keys` agar widget tidak ikut di-rebuild saat key lain berubah:

```
ValueListenableBuilder<Box>(
  valueListenable: Hive.box('pengaturan').listenable(keys: ['tema_gelap']),
  builder: (context, box, _) {
    final gelap = box.get('tema_gelap', defaultValue: false) as bool;
    return Icon(gelap ? Icons.dark_mode : Icons.light_mode);
  },
)
```

> 💡 **Kapan cukup ValueListenableBuilder, kapan butuh controller GetX?**
>
> `ValueListenableBuilder` sangat praktis untuk layar sederhana yang langsung menampilkan isi box: widget mendengarkan **box** secara langsung. Ketika ada logika tambahan (validasi, pencarian, pengurutan, gabungan beberapa sumber data), lebih rapi jika logika itu dipindah ke service dan `GetxController` seperti di Modul 08. Bedanya, dengan GetX widget tidak lagi mendengarkan box, melainkan nilai `.obs` di controller lewat `Obx`, dan view tidak perlu tahu bahwa data berasal dari Hive. Itu topik bagian berikutnya.

8. Hive + GetX Controller (Pola MVC)

Di Modul 08 kamu belajar GetX dengan pola MVC untuk mengelola state. Sekarang kita gabungkan dengan Hive: **service** mengurus penyimpanan di box, **controller** (`GetxController`) menyimpan state reaktif, dan **view** cukup menampilkan data lewat `Obx`.

Pembagian tugas

Lapisan	Folder	Tugas	Boleh tahu tentang
Model	`models/`	Class data murni (bagian 5 dan 6)	Tidak bergantung pada apa pun
Service	`services/`	Membaca dan menulis box Hive	Hive, model
Controller	`controllers/`	Logika aplikasi dan state `.obs`, memanggil service	Service, model (tanpa widget, tanpa Hive langsung)
View	`views/`	Menampilkan data dengan `Obx`, meneruskan aksi pengguna ke controller	Controller, model (tidak menyentuh Hive langsung)

Arah ketergantungannya satu jalur: **View → Controller → Service → Hive**. Keuntungannya: jika suatu saat penyimpanan diganti (misalnya ke Supabase di Modul 12), yang diubah cukup service. View tidak perlu disentuh, dan service maupun controller mudah diuji tanpa UI.

> ℹ️ **Service atau repository?**
>
> Di banyak tutorial, kelas yang membungkus box Hive disebut *repository*. Di kursus ini kita memakai nama **service** agar sama dengan struktur MVC Modul 08. Isinya sama saja: satu-satunya tempat yang menyentuh box. Di Modul 13–14 kamu akan melihat lagi nama repository untuk lapisan yang serupa.

Contoh: pengaturan tema gelap dan nama pengguna yang tersimpan di Hive. Pasang package dulu: `flutter pub add hive_ce hive_ce_flutter get`.

Langkah 1: Service

`lib/services/pengaturan_service.dart`

```dart
// lib/services/pengaturan_service.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

class PengaturanService {
  static const String namaBox = 'pengaturan';
  static const String _kunciTemaGelap = 'tema_gelap';
  static const String _kunciNama = 'nama_pengguna';

  final Box _box;

  PengaturanService(this._box);

  static Future<PengaturanService> buka() async {
    final box = await Hive.openBox(namaBox);
    return PengaturanService(box);
  }

  bool get temaGelap => _box.get(_kunciTemaGelap, defaultValue: false) as bool;
  Future<void> setTemaGelap(bool nilai) => _box.put(_kunciTemaGelap, nilai);

  String get namaPengguna =>
      _box.get(_kunciNama, defaultValue: 'Mahasiswa') as String;
  Future<void> setNamaPengguna(String nama) => _box.put(_kunciNama, nama);
}
```

Nama box dan nama key disimpan sebagai konstanta privat. Seluruh aplikasi tidak perlu tahu bahwa tema disimpan dengan key `'tema_gelap'`, sehingga salah ketik key tidak tersebar di banyak file. Perhatikan juga bahwa service tidak mengimpor GetX sama sekali: ia hanya Dart + Hive.

Langkah 2: Controller

`lib/controllers/pengaturan_controller.dart`

```dart
// lib/controllers/pengaturan_controller.dart
import 'package:get/get.dart';

import '../services/pengaturan_service.dart';

class PengaturanController extends GetxController {
  PengaturanController(this._service);
  final PengaturanService _service;

  // State reaktif privat
  final _temaGelap = false.obs;
  final _namaPengguna = ''.obs;

  // Getter publik read-only
  bool get temaGelap => _temaGelap.value;
  String get namaPengguna => _namaPengguna.value;

  @override
```

```
  void onInit() {
    super.onInit();
    // Nilai awal langsung dibaca dari Hive (sinkron, tanpa await)
    _temaGelap.value = _service.temaGelap;
    _namaPengguna.value = _service.namaPengguna;
  }

  Future<void> gantiTema(bool gelap) async {
    await _service.setTemaGelap(gelap); // simpan permanen dulu
    _temaGelap.value = gelap; // lalu kabari Obx
  }

  Future<void> gantiNama(String nama) async {
    await _service.setNamaPengguna(nama);
    _namaPengguna.value = nama;
  }
}
```

- Service masuk lewat constructor, persis seperti pola controller di Modul 08. Controller tidak tahu cara membuka box.
- Nilai awal diisi di `onInit()`. Membaca box Hive itu sinkron, jadi tidak perlu `isLoading` atau `await`.
- Setiap method tulis menyimpan ke Hive dulu, baru mengubah nilai `.obs`. Mengubah `.value` otomatis membuat setiap `Obx` yang membacanya dibangun ulang.

Langkah 3: main.dart dan view

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'controllers/pengaturan_controller.dart';
import 'services/pengaturan_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  final service = await PengaturanService.buka();
  Get.put(PengaturanController(service));

  runApp(const MyApp());
}

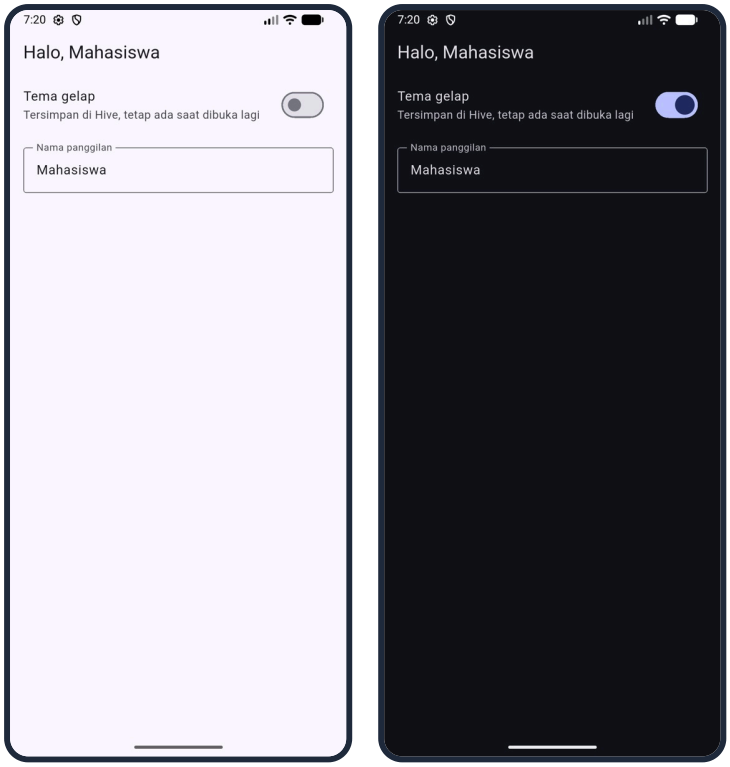
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final controller = Get.find<PengaturanController>();
    // Obx membangun ulang GetMaterialApp hanya saat temaGelap berubah
    return Obx(
      () => GetMaterialApp(
```

```
        debugShowCheckedModeBanner: false,
        theme: ThemeData(colorSchemeSeed: Colors.indigo),
        darkTheme: ThemeData(
          colorSchemeSeed: Colors.indigo,
          brightness: Brightness.dark,
        ),
        themeMode: controller.temaGelap ? ThemeMode.dark : ThemeMode.light,
        home: const PengaturanView(),
      ),
    );
  }
}

class PengaturanView extends GetView<PengaturanController> {
  const PengaturanView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Obx(() => Text('Halo, ${controller.namaPengguna}')),
      ),
      body: ListView(
        children: [
          Obx(
            () => SwitchListTile(
              title: const Text('Tema gelap'),
              subtitle:
                  const Text('Tersimpan di Hive, tetap ada saat dibuka lagi'),
              value: controller.temaGelap,
              onChanged: controller.gantiTema,
            ),
          ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: TextFormField(
              initialValue: controller.namaPengguna,
              decoration: const InputDecoration(
                labelText: 'Nama panggilan',
                border: OutlineInputBorder(),
              ),
              onFieldSubmitted: (nama) => controller.gantiNama(nama.trim()),
            ),
          ),
        ],
      ),
    );
  }
}
```



Tema terang (nilai awal)

Setelah switch ditekan: tema gelap tersimpan dan tetap aktif saat aplikasi dibuka lagi

Penjelasan:

- Service dibuka di `main()` (async), lalu diberikan ke controller lewat constructor dan didaftarkan dengan `Get.put`. Dengan begitu controller tidak perlu berurusan dengan `await openBox`.
- `Obx` di `MyApp` membaca `controller.temaGelap`, sehingga `GetMaterialApp` dibangun ulang hanya saat tema berubah.
- `PengaturanView` memakai `GetView<PengaturanController>` sehingga getter `controller` langsung tersedia. Hanya judul AppBar dan switch yang dibungkus `Obx`, karena hanya bagian itu yang perlu berubah.
- Di callback (`onChanged`, `onFieldSubmitted`) controller dipanggil langsung tanpa `Obx`, aturan yang sama dengan Modul 08.
- `TextFormField` memakai `initialValue` yang hanya dibaca sekali, jadi tidak perlu dibungkus `Obx`.

> ℹ️ **Bandingkan dengan Modul 10**
>
> Di SharedPreferences, membaca nilai butuh `await SharedPreferences.getInstance()`, sehingga tema awal harus dimuat secara async. Dengan Hive, setelah box dibuka di `main()`, nilai bisa langsung dibaca di `onInit()` controller tanpa menunggu.

⚠️ **Kesalahan umum**

- Memanggil `Hive.box(...)` langsung dari view. Biarkan hanya service yang tahu tentang box.
- Mengubah data di box tanpa mengubah `.obs` di controller. Box memang berubah, tetapi `Obx` tidak tahu dan layar tetap menampilkan nilai lama.
- Membungkus seluruh `Scaffold` dengan satu `Obx` besar. Bungkus hanya widget yang membaca nilai reaktif.

9. Enkripsi Box

File box Hive tersimpan sebagai data biner biasa. Siapa pun yang bisa menyalin file aplikasi (misalnya di HP yang di-root) bisa membaca isinya. Untuk data sensitif seperti token atau catatan pribadi, Hive mendukung **enkripsi AES-256** per box.

Bahan yang dibutuhkan

- **Kunci 256-bit** (32 byte). Dibuat sekali dengan `Hive.generateSecureKey()`.
- **Tempat aman untuk kunci.** Kunci tidak boleh disimpan di Hive biasa atau ditulis di kode. Simpan di `flutter_secure_storage`, yang memakai Keystore (Android) dan Keychain (iOS).
- `HiveAesCipher(kunci)` yang diberikan ke `openBox` lewat parameter `encryptionCipher`.

```
flutter pub add flutter_secure_storage
```

Versi terbaru `flutter_secure_storage` membutuhkan Android minimal API 23. Jika build gagal karena `minSdk`, ubah `minSdk` di `android/app/build.gradle.kts` menjadi minimal 23.

`lib/services/box_rahasia.dart`

```dart
// lib/services/box_rahasia.dart
import 'dart:convert';

import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

const _storage = FlutterSecureStorage();
const _namaKunci = 'kunci_enkripsi_hive';

Future<Box<String>> bukaBoxRahasia() async {
  // 1. Coba ambil kunci yang pernah disimpan
  var kunciBase64 = await _storage.read(key: _namaKunci);

  // 2. Belum ada? Buat kunci acak 256-bit lalu simpan ke secure storage
  if (kunciBase64 == null) {
    final kunciBaru = Hive.generateSecureKey(); // List<int> 32 byte
    kunciBase64 = base64UrlEncode(kunciBaru);
    await _storage.write(key: _namaKunci, value: kunciBase64);
  }

  // 3. Buka box dengan cipher AES
  final kunci = base64Url.decode(kunciBase64);
  return Hive.openBox<String>(
    'rahasia',
    encryptionCipher: HiveAesCipher(kunci),
  );
}
```

Pemakaiannya sama seperti box biasa. Enkripsi dan dekripsi terjadi otomatis:

```dart
// Di main(), setelah await Hive.initFlutter();
final rahasia = await bukaBoxRahasia();
await rahasia.put('token', 'eyJhbGciOi...');
debugPrint(rahasia.get('token')); // eyJhbGciOi...
```

Penjelasan:

- Kunci berupa `List<int>`, sedangkan secure storage hanya menyimpan String. Karena itu kunci diubah dengan `base64UrlEncode` saat disimpan dan `base64Url.decode` saat dipakai.
- Kunci dibuat sekali saja. Membuat kunci baru setiap aplikasi dibuka berarti data lama tidak akan bisa dibaca lagi.
- Hanya **value** yang dienkripsi. **Key** box tetap tersimpan tanpa enkripsi, jadi jangan menaruh informasi sensitif di nama key.

> ⚠️ **Perhatian**
>
> - Membuka box terenkripsi dengan kunci yang salah, atau membuka box biasa yang sudah berisi data dengan cipher, akan memicu error. Tentukan sejak awal box mana yang dienkripsi.
> - Jika kunci hilang (misalnya data secure storage terhapus), isi box tidak bisa dipulihkan. Untuk data yang masih ada di server, cukup hapus box lalu unduh ulang.

10. Mini Project: Catatan Kuliah Offline (CRUD Lengkap)

Saatnya merangkai semua materi menjadi satu aplikasi utuh: **Catatan Kuliah Offline**. Aplikasi ini berjalan 100% tanpa internet dan datanya tetap ada walaupun aplikasi ditutup.

Fitur yang dibangun

- **Create:** menambah catatan (judul, mata kuliah, isi) lewat form dengan validasi.
- **Read:** menampilkan daftar catatan dalam `ListView`.
- **Update:** mengetuk catatan untuk membuka form edit yang sudah terisi.
- **Delete:** menghapus catatan setelah konfirmasi dialog.
- **Cari:** berdasarkan judul, mata kuliah, atau isi.
- **Urutkan:** terbaru, terlama, atau judul A-Z.
- **Empty state:** tampilan khusus saat belum ada catatan dan saat pencarian tidak menemukan hasil.

Arsitektur

```
View (HomeView, FormCatatanView)        ← V: tampilan, Obx, navigasi, SnackBar
   |  Obx(() => controller.daftar) / controller.tambah(...)
   ▼
CatatanController (GetxController)      ← C: state .obs, cari, urutkan, tambah, ubah, hapus
   |
   ▼
CatatanService                          ← satu-satunya yang menyentuh Hive
   |
   ▼
Box<Catatan> 'catatan_box'  +  CatatanAdapter (typeId 0)
```

Struktur folder

```
catatan_kuliah/
├── lib/
|   ├── main.dart
|   ├── models/
|   |   ├── catatan.dart
|   |   └── catatan_adapter.dart
|   ├── services/
|   |   └── catatan_service.dart
|   ├── controllers/
|   |   └── catatan_controller.dart
|   ├── views/
|   |   ├── home_view.dart
|   |   └── form_catatan_view.dart
|   └── widgets/
|       ├── catatan_tile.dart
|       └── empty_state.dart
└── test/
    └── catatan_service_test.dart   (bonus)
```

Ini struktur MVC yang sama dengan Modul 08: **M**odel di `models/`, **V**iew di `views/`, **C**ontroller di `controllers/`, ditambah `services/` untuk akses Hive dan `widgets/` untuk komponen kecil.

> 💡 **Urutan membangun**
>
> Kerjakan dari lapisan paling bawah ke atas: model, adapter, service, controller, widget kecil, view, lalu `main.dart`. Dengan urutan ini setiap file yang kamu tulis hanya bergantung pada file yang sudah ada, sehingga error merah di editor cepat hilang.

Langkah 0: Buat proyek dan pasang package

```
flutter create catatan_kuliah
cd catatan_kuliah
flutter pub add hive_ce hive_ce_flutter get
```

Hapus file `test/widget_test.dart` bawaan karena isinya menguji aplikasi counter yang akan kita ganti.

Langkah 1: Model `Catatan`

Model berisi data murni tanpa kode Hive. `copyWith` dipakai saat edit agar kita tidak mengubah objek lama, melainkan membuat salinan dengan field yang diganti. `id` dan `dibuatPada` sengaja tidak bisa diganti.

📄 **`lib/models/catatan.dart`**

```dart
// lib/models/catatan.dart
class Catatan {
  final String id;
  final String judul;
  final String mataKuliah;
  final String isi;
  final DateTime dibuatPada;
  final DateTime diubahPada;

  const Catatan({
    required this.id,
    required this.judul,
    required this.mataKuliah,
    required this.isi,
    required this.dibuatPada,
    required this.diubahPada,
  });

  // Membuat salinan objek dengan sebagian field diganti (dipakai saat edit)
  Catatan copyWith({
    String? judul,
    String? mataKuliah,
    String? isi,
```

```
    DateTime? diubahPada,
  }) {
    return Catatan(
      id: id,
      judul: judul ?? this.judul,
      mataKuliah: mataKuliah ?? this.mataKuliah,
      isi: isi ?? this.isi,
      dibuatPada: dibuatPada,
      diubahPada: diubahPada ?? this.diubahPada,
    );
  }
}
```

Langkah 2: TypeAdapter manual

Adapter ini menerapkan pelajaran dari bagian 5: urutan baca sama dengan urutan tulis, `DateTime` disimpan sebagai angka milidetik, dan byte pertama adalah nomor versi format agar migrasi di masa depan aman.

lib/models/catatan_adapter.dart

```dart
// lib/models/catatan_adapter.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'catatan.dart';

class CatatanAdapter extends TypeAdapter<Catatan> {
  // typeId harus unik di seluruh aplikasi dan TIDAK BOLEH diganti setelah rilis
  @override
  final int typeId = 0;

  // Versi format penyimpanan. Naikkan jika suatu saat menambah field baru.
  static const int _versi = 1;

  @override
  Catatan read(BinaryReader reader) {
    final versi = reader.readByte(); // dibaca dulu, dipakai saat migrasi nanti
    assert(versi <= _versi, 'Data dibuat oleh versi aplikasi yang lebih baru');
    // Urutan baca HARUS sama persis dengan urutan tulis di write()
    return Catatan(
      id: reader.readString(),
      judul: reader.readString(),
      mataKuliah: reader.readString(),
      isi: reader.readString(),
      dibuatPada: DateTime.fromMillisecondsSinceEpoch(reader.readInt()),
      diubahPada: DateTime.fromMillisecondsSinceEpoch(reader.readInt()),
    );
  }

  @override
  void write(BinaryWriter writer, Catatan obj) {
    writer.writeByte(_versi);
    writer.writeString(obj.id);
    writer.writeString(obj.judul);
    writer.writeString(obj.mataKuliah);
    writer.writeString(obj.isi);
    writer.writeInt(obj.dibuatPada.millisecondsSinceEpoch);
```

```
    writer.writeInt(obj.diubahPada.millisecondsSinceEpoch);
  }
}
```

Langkah 3: Service

Setiap catatan disimpan dengan `put(catatan.id, catatan)`. Karena key-nya adalah id catatan, menyimpan catatan yang sudah ada otomatis menjadi update, dan menghapus cukup dengan id. Kita tidak memakai index sama sekali.

lib/services/catatan_service.dart

```dart
// lib/services/catatan_service.dart
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import '../models/catatan.dart';

/// Satu-satunya kelas yang boleh menyentuh box Hive catatan.
/// Controller cukup memanggil method di sini; view tidak pernah menyentuh Hive.
class CatatanService {
  static const String namaBox = 'catatan_box';

  final Box<Catatan> _box;

  CatatanService(this._box);

  /// Membuka box lalu membungkusnya menjadi service.
  /// Pastikan Hive sudah di-init dan CatatanAdapter sudah didaftarkan.
  static Future<CatatanService> buka() async {
    final box = await Hive.openBox<Catatan>(namaBox);
    return CatatanService(box);
  }

  /// READ: semua catatan (urutan belum diatur)
  List<Catatan> ambilSemua() => _box.values.toList();

  /// READ: satu catatan berdasarkan id (null jika tidak ada)
  Catatan? ambil(String id) => _box.get(id);

  int get jumlah => _box.length;

  /// CREATE & UPDATE: put() dengan key yang sama akan menimpa data lama
  Future<void> simpan(Catatan catatan) => _box.put(catatan.id, catatan);

  /// DELETE: hapus satu catatan
  Future<void> hapus(String id) => _box.delete(id);

  /// DELETE: kosongkan seluruh catatan
  Future<int> hapusSemua() => _box.clear();
}
```

Langkah 4: Controller

Controller menyimpan state reaktif: salinan isi box (`_semua`), kata kunci pencarian, dan pilihan urutan. Getter `daftar` menghasilkan data yang sudah difilter dan diurutkan. Setiap operasi tulis

memanggil service, lalu memuat ulang `_semua` dari box sehingga semua `Obx` yang membacanya ikut diperbarui.

lib/controllers/catatan_controller.dart

```dart
// lib/controllers/catatan_controller.dart
import 'package:get/get.dart';

import '../models/catatan.dart';
import '../services/catatan_service.dart';

enum UrutanCatatan { terbaru, terlama, judulAZ }

class CatatanController extends GetxController {
  CatatanController(this._service);
  final CatatanService _service;

  // State privat reaktif
  final _semua = <Catatan>[].obs; // salinan isi box
  final _kataKunci = ''.obs;
  final _urutan = UrutanCatatan.terbaru.obs;

  // Getter publik read-only
  String get kataKunci => _kataKunci.value;
  UrutanCatatan get urutan => _urutan.value;

  /// Jumlah seluruh catatan (tanpa filter pencarian)
  int get totalCatatan => _semua.length;

  /// Daftar yang siap ditampilkan: sudah difilter lalu diurutkan
  List<Catatan> get daftar {
    final kunci = _kataKunci.value.trim().toLowerCase();
    final hasil = _semua.where((c) {
      if (kunci.isEmpty) return true;
      return c.judul.toLowerCase().contains(kunci) ||
          c.mataKuliah.toLowerCase().contains(kunci) ||
          c.isi.toLowerCase().contains(kunci);
    }).toList();

    switch (_urutan.value) {
      case UrutanCatatan.terbaru:
        hasil.sort((a, b) => b.diubahPada.compareTo(a.diubahPada));
      case UrutanCatatan.terlama:
        hasil.sort((a, b) => a.diubahPada.compareTo(b.diubahPada));
      case UrutanCatatan.judulAZ:
        hasil.sort(
            (a, b) => a.judul.toLowerCase().compareTo(b.judul.toLowerCase()));
    }
    return hasil;
  }

  @override
  void onInit() {
    super.onInit();
    _muatUlang(); // membaca box itu sinkron, jadi tidak perlu isLoading
  }

  void _muatUlang() => _semua.assignAll(_service.ambilSemua());
```

```dart
  void cari(String kataKunci) => _kataKunci.value = kataKunci;

  void ubahUrutan(UrutanCatatan urutan) => _urutan.value = urutan;

  Future<void> tambah({
    required String judul,
    required String mataKuliah,
    required String isi,
  }) async {
    final sekarang = DateTime.now();
    final catatan = Catatan(
      // id unik sederhana berbasis waktu (tanpa package tambahan)
      id: sekarang.microsecondsSinceEpoch.toString(),
      judul: judul,
      mataKuliah: mataKuliah,
      isi: isi,
      dibuatPada: sekarang,
      diubahPada: sekarang,
    );
    await _service.simpan(catatan);
    _muatUlang();
  }

  Future<void> ubah(
    Catatan lama, {
    required String judul,
    required String mataKuliah,
    required String isi,
  }) async {
    final baru = lama.copyWith(
      judul: judul,
      mataKuliah: mataKuliah,
      isi: isi,
      diubahPada: DateTime.now(),
    );
    await _service.simpan(baru); // key sama, data lama tertimpa
    _muatUlang();
  }

  Future<void> hapus(String id) async {
    await _service.hapus(id);
    _muatUlang();
  }
}
```

- File ini hanya mengimpor `package:get/get.dart`, model, dan service. Tidak ada `material.dart`, `BuildContext`, SnackBar, maupun navigasi; semua itu urusan view. Karena itu controller bisa diuji dengan unit test biasa.
- `_muatUlang()` memakai `assignAll` untuk mengganti seluruh isi `RxList` sekaligus. Dipanggil di `onInit()` dan setelah setiap operasi tulis.
- `daftar` adalah **data turunan** berbentuk getter biasa. Karena getter ini membaca `_semua`, `_kataKunci.value`, dan `_urutan.value`, `Obx` yang memanggilnya otomatis mendengarkan ketiganya.

- `switch` tanpa `break` adalah sintaks Dart 3: setiap `case` otomatis berhenti.
- `totalCatatan` dipisah dari `daftar.length` agar layar bisa membedakan "belum ada catatan sama sekali" dan "pencarian tidak menemukan apa pun".
- Id dibuat dari `microsecondsSinceEpoch` sehingga cukup unik untuk satu perangkat tanpa package tambahan. Di Modul 14 kita akan memakai id dari server.

Langkah 5: Widget kecil yang dipakai ulang

lib/widgets/empty_state.dart

```dart
// lib/widgets/empty_state.dart
import 'package:flutter/material.dart';

class EmptyState extends StatelessWidget {
  final IconData ikon;
  final String judul;
  final String pesan;

  const EmptyState({
    super.key,
    required this.ikon,
    required this.judul,
    required this.pesan,
  });

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(ikon, size: 72, color: warna.primary.withValues(alpha: 0.5)),
            const SizedBox(height: 16),
            Text(
              judul,
              style: Theme.of(context).textTheme.titleMedium,
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 8),
            Text(
              pesan,
              style: TextStyle(color: warna.onSurfaceVariant),
              textAlign: TextAlign.center,
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/widgets/catatan_tile.dart

```dart
// lib/widgets/catatan_tile.dart
import 'package:flutter/material.dart';

import '../models/catatan.dart';

class CatatanTile extends StatelessWidget {
  final Catatan catatan;
  final VoidCallback onTap;
  final VoidCallback onHapus;

  const CatatanTile({
    super.key,
    required this.catatan,
    required this.onTap,
    required this.onHapus,
  });

  String _formatTanggal(DateTime t) {
    String duaDigit(int n) => n.toString().padLeft(2, '0');
    return '${duaDigit(t.day)}/${duaDigit(t.month)}/${t.year} '
        '${duaDigit(t.hour)}:${duaDigit(t.minute)}';
  }

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        onTap: onTap,
        leading: CircleAvatar(
          backgroundColor: warna.primaryContainer,
          child: Text(
            catatan.mataKuliah.isEmpty
                ? '?'
                : catatan.mataKuliah[0].toUpperCase(),
          ),
        ),
        title: Text(
          catatan.judul,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(catatan.mataKuliah, style: TextStyle(color: warna.primary)),
            if (catatan.isi.isNotEmpty)
              Text(catatan.isi, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 4),
            Text(
              'Diubah: ${_formatTanggal(catatan.diubahPada)}',
              style: Theme.of(context).textTheme.bodySmall,
            ),
          ],
        ),
```

```
        isThreeLine: true,
        trailing: IconButton(
          icon: const Icon(Icons.delete_outline),
          tooltip: 'Hapus',
          onPressed: onHapus,
        ),
      ),
    );
  }
}
```

`CatatanTile` tidak tahu apa pun tentang GetX, controller, atau Hive. Ia hanya menerima data dan dua callback (`onTap`, `onHapus`), sehingga mudah dipakai ulang dan diuji.

Langkah 6: View form (tambah & edit)

Satu layar dipakai untuk dua mode. Jika parameter `catatan` bernilai `null`, form berada di mode tambah; jika berisi, form di mode edit dan field langsung terisi data lama.

lib/views/form_catatan_view.dart

```dart
// lib/views/form_catatan_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/catatan_controller.dart';
import '../models/catatan.dart';

class FormCatatanView extends StatefulWidget {
  /// null = mode tambah, berisi = mode edit
  final Catatan? catatan;

  const FormCatatanView({super.key, this.catatan});

  @override
  State<FormCatatanView> createState() => _FormCatatanViewState();
}

class _FormCatatanViewState extends State<FormCatatanView> {
  // Controller yang sama dengan yang dipakai HomeView
  final _controller = Get.find<CatatanController>();

  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judulCtrl;
  late final TextEditingController _matkulCtrl;
  late final TextEditingController _isiCtrl;
  bool _menyimpan = false;

  bool get _modeEdit => widget.catatan != null;

  @override
  void initState() {
    super.initState();
    // Saat mode edit, isi form dengan data lama
    _judulCtrl = TextEditingController(text: widget.catatan?.judul ?? '');
    _matkulCtrl = TextEditingController(text: widget.catatan?.mataKuliah ?? '');
    _isiCtrl = TextEditingController(text: widget.catatan?.isi ?? '');
```

```dart
  }

  @override
  void dispose() {
    _judulCtrl.dispose();
    _matkulCtrl.dispose();
    _isiCtrl.dispose();
    super.dispose();
  }

  String? _wajibDiisi(String? nilai) {
    if (nilai == null || nilai.trim().isEmpty) return 'Wajib diisi';
    return null;
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    setState(() => _menyimpan = true);

    final judul = _judulCtrl.text.trim();
    final matkul = _matkulCtrl.text.trim();
    final isi = _isiCtrl.text.trim();

    if (_modeEdit) {
      await _controller.ubah(
        widget.catatan!,
        judul: judul,
        mataKuliah: matkul,
        isi: isi,
      );
    } else {
      await _controller.tambah(judul: judul, mataKuliah: matkul, isi: isi);
    }

    if (!mounted) return; // widget bisa saja sudah ditutup saat menunggu
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(_modeEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
      ),
    );
    Get.back();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(_modeEdit ? 'Edit Catatan' : 'Catatan Baru'),
      ),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judulCtrl,
              decoration: const InputDecoration(
                labelText: 'Judul',
                border: OutlineInputBorder(),
```

```
              ),
              textInputAction: TextInputAction.next,
              validator: _wajibDiisi,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _matkulCtrl,
              decoration: const InputDecoration(
                labelText: 'Mata Kuliah',
                hintText: 'mis. Pemrograman Mobile',
                border: OutlineInputBorder(),
              ),
              textInputAction: TextInputAction.next,
              validator: _wajibDiisi,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isiCtrl,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                alignLabelWithHint: true,
                border: OutlineInputBorder(),
              ),
              minLines: 6,
              maxLines: 12,
            ),
            const SizedBox(height: 20),
            FilledButton.icon(
              onPressed: _menyimpan ? null : _simpan,
              icon: const Icon(Icons.save),
              label: Text(_modeEdit ? 'Simpan Perubahan' : 'Simpan'),
            ),
          ],
        ),
      ),
    );
  }
}
```

- `Get.find<CatatanController>()` mengambil controller yang sama yang sudah didaftarkan di `main()`. Tidak perlu mengoper controller lewat constructor atau `BuildContext`.
- `TextEditingController` dan status `_menyimpan` tetap disimpan di `State` karena hanya dipakai di layar ini (state lokal, seperti di Modul 05).
- Controller hanya menyimpan data. Yang menampilkan SnackBar dan menutup layar (`Get.back()`) adalah view.
- `if (!mounted) return;` setelah `await` mencegah error jika layar sudah ditutup ketika penyimpanan selesai.
- Tombol simpan dinonaktifkan (`onPressed: null`) selama menyimpan agar tidak terjadi simpan ganda.

Langkah 7: View utama

`lib/views/home_view.dart`

```dart
// lib/views/home_view.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/catatan_controller.dart';
import '../models/catatan.dart';
import '../widgets/catatan_tile.dart';
import '../widgets/empty_state.dart';
import 'form_catatan_view.dart';

// GetView<T> menyediakan getter `controller` (= Get.find<T>())
class HomeView extends GetView<CatatanController> {
  const HomeView({super.key});

  void _bukaForm([Catatan? catatan]) {
    Get.to(() => FormCatatanView(catatan: catatan));
  }

  Future<void> _konfirmasiHapus(BuildContext context, Catatan catatan) async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (dialogContext) => AlertDialog(
        title: const Text('Hapus catatan?'),
        content: Text('Catatan "${catatan.judul}" akan dihapus permanen.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(dialogContext, false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(dialogContext, true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );

    // showDialog mengembalikan null jika dialog ditutup dengan tap di luar
    if (yakin != true) return;

    await controller.hapus(catatan.id);
    if (!context.mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('"${catatan.judul}" dihapus')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Kuliah'),
        actions: [
          Obx(
```

```
            () => PopupMenuButton<UrutanCatatan>(
              icon: const Icon(Icons.sort),
              tooltip: 'Urutkan',
              initialValue: controller.urutan,
              onSelected: controller.ubahUrutan,
              itemBuilder: (_) => const [
                PopupMenuItem(
                  value: UrutanCatatan.terbaru,
                  child: Text('Terbaru dulu'),
                ),
                PopupMenuItem(
                  value: UrutanCatatan.terlama,
                  child: Text('Terlama dulu'),
                ),
                PopupMenuItem(
                  value: UrutanCatatan.judulAZ,
                  child: Text('Judul A-Z'),
                ),
              ],
            ),
          ),
        ],
      ),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(12, 8, 12, 4),
            child: TextField(
              decoration: const InputDecoration(
                hintText: 'Cari judul, mata kuliah, atau isi...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
                isDense: true,
              ),
              onChanged: controller.cari,
            ),
          ),
          // Hanya bagian daftar yang dibangun ulang saat state berubah
          Expanded(child: Obx(() => _buildDaftar(context))),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _bukaForm,
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
    );
  }

  Widget _buildDaftar(BuildContext context) {
    final daftar = controller.daftar;

    if (controller.totalCatatan == 0) {
      return const EmptyState(
        ikon: Icons.note_alt_outlined,
        judul: 'Belum ada catatan',
        pesan: 'Tekan tombol + Catatan untuk menulis catatan kuliah pertamamu.',
      );
    }
```

```
    if (daftar.isEmpty) {
      return EmptyState(
        ikon: Icons.search_off,
        judul: 'Tidak ditemukan',
        pesan: 'Tidak ada catatan yang cocok dengan "${controller.kataKunci}".',
      );
    }
    return ListView.builder(
      padding: const EdgeInsets.only(bottom: 88), // agar tidak tertutup FAB
      itemCount: daftar.length,
      itemBuilder: (context, index) {
        final catatan = daftar[index];
        return CatatanTile(
          key: ValueKey(catatan.id),
          catatan: catatan,
          onTap: () => _bukaForm(catatan),
          onHapus: () => _konfirmasiHapus(context, catatan),
        );
      },
    );
  }
}
```

- `HomeView extends GetView<CatatanController>` menyediakan getter `controller`, sehingga tidak perlu menulis `Get.find` berulang kali.
- Hanya dua bagian yang dibungkus `Obx`: menu urutkan (membaca `urutan`) dan daftar catatan (membaca `daftar`, `totalCatatan`, `kataKunci`). Kolom pencarian dan FAB tidak ikut dibangun ulang saat mengetik.
- Callback seperti `onChanged: controller.cari` dan `onSelected: controller.ubahUrutan` memanggil controller langsung tanpa `Obx`.
- Navigasi ke form memakai `Get.to(() => FormCatatanView(...))`. Saat form ditutup, daftar sudah terbarui karena controller memuat ulang data setelah menyimpan.
- `showDialog<bool>` mengembalikan `true` / `false` dari tombol, atau `null` jika dialog ditutup dengan mengetuk area luar. Karena itu yang dicek adalah `yakin != true`.
- `context.mounted` dicek setelah `await controller.hapus(...)` sebelum `context` dipakai lagi untuk SnackBar.
- `ValueKey(catatan.id)` membantu Flutter mengenali item yang sama saat urutan list berubah.

Langkah 8: `main.dart`

`lib/main.dart`

```
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'controllers/catatan_controller.dart';
import 'models/catatan_adapter.dart';
import 'services/catatan_service.dart';
```

```dart
import 'views/home_view.dart';

Future<void> main() async {
  // 1. Wajib sebelum memakai plugin (path_provider) sebelum runApp
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Siapkan folder penyimpanan Hive
  await Hive.initFlutter();

  // 3. Daftarkan adapter SEBELUM membuka box yang memakainya
  Hive.registerAdapter(CatatanAdapter());

  // 4. Buka box sekali di awal, lalu bungkus dengan service
  final service = await CatatanService.buka();

  // 5. Daftarkan controller agar bisa diambil dengan Get.find di view mana pun
  Get.put(CatatanController(service));

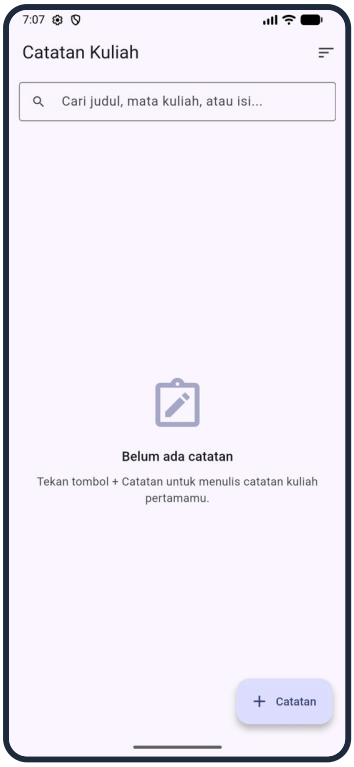
  runApp(const CatatanApp());
}

class CatatanApp extends StatelessWidget {
  const CatatanApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Catatan Kuliah Offline',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const HomeView(),
    );
  }
}
```

Langkah 9: Jalankan dan uji

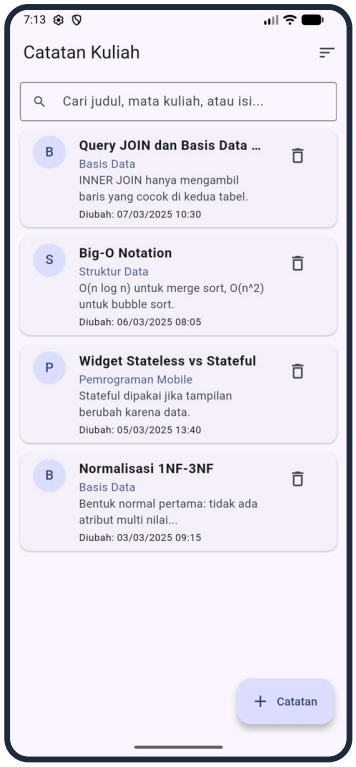
```
flutter run
```



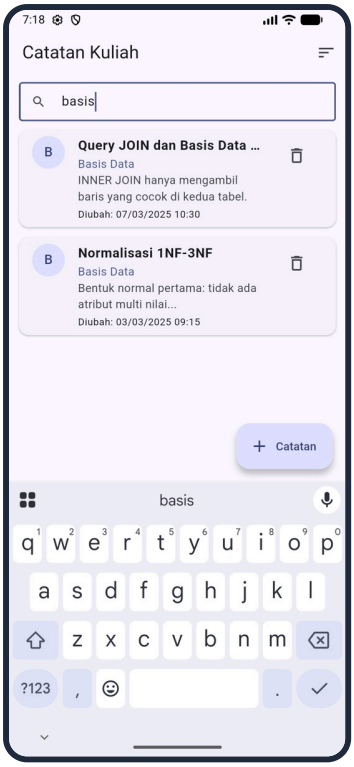
Empty state saat box masih kosong



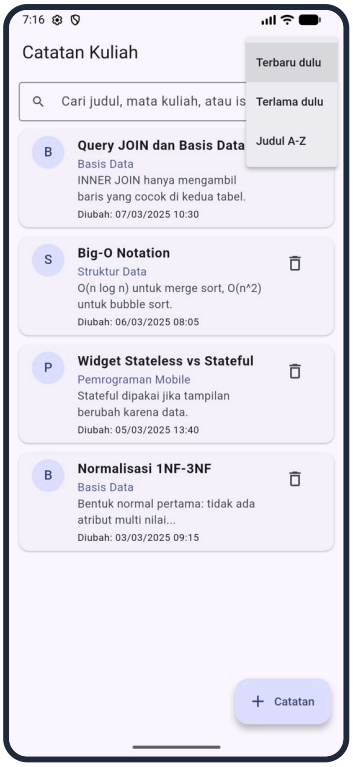
Form tambah catatan



Daftar catatan, terbaru di atas



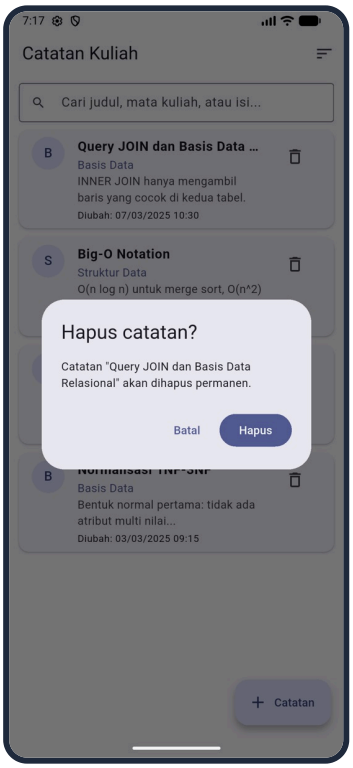
Pencarian kata "basis"



Menu urutkan di AppBar



Mode edit: form terisi data lama



Konfirmasi sebelum menghapus

Daftar uji manual:

1. Buka aplikasi pertama kali: muncul tulisan "Belum ada catatan".
2. Tekan **+ Catatan**, lalu langsung tekan Simpan: muncul pesan "Wajib diisi" pada judul dan mata kuliah.
3. Isi form lalu simpan: catatan muncul di daftar dan SnackBar tampil.

4. Tambah minimal 3 catatan, lalu coba ketiga pilihan urutan.
5. Ketik kata kunci di kolom cari; coba juga kata yang tidak ada untuk melihat empty state "Tidak ditemukan".
6. Ketuk sebuah catatan, ubah judulnya, lalu simpan. Catatan itu naik ke atas karena `diubahPada` diperbarui.
7. Tekan ikon hapus, pilih Batal (data tetap ada), lalu ulangi dan pilih Hapus.
8. **Tutup aplikasi sepenuhnya** (bukan hot reload), lalu buka lagi: semua catatan masih ada.

Bonus: unit test service

Service yang terpisah dari UI bisa diuji tanpa emulator. Di unit test tidak ada `path_provider`, sehingga kita memakai `Hive.init(folder)` dengan folder sementara sebagai pengganti `Hive.initFlutter()`. Ganti `catatan_kuliah` pada baris import jika nama proyekmu berbeda (lihat `name:` di `pubspec.yaml`).

`test/catatan_service_test.dart`

```dart
// test/catatan_service_test.dart
import 'dart:io';

import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:catatan_kuliah/models/catatan.dart';
import 'package:catatan_kuliah/models/catatan_adapter.dart';
import 'package:catatan_kuliah/services/catatan_service.dart';

void main() {
  late Directory tempDir;
  late CatatanService service;

  setUp(() async {
    // Di unit test tidak ada path_provider, jadi pakai Hive.init + folder sementara
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    if (!Hive.isAdapterRegistered(0)) {
      Hive.registerAdapter(CatatanAdapter());
    }
    service = await CatatanService.buka();
  });

  tearDown(() async {
    await Hive.deleteFromDisk(); // hapus semua box
    await Hive.close();
    await tempDir.delete(recursive: true);
  });

  Catatan buatCatatan(String id, String judul) {
    final t = DateTime(2025, 1, 1, 8, 0);
    return Catatan(
      id: id,
      judul: judul,
      mataKuliah: 'Pemrograman Mobile',
      isi: 'Isi $judul',
```

```
      dibuatPada: t,
      diubahPada: t,
    );
  }

  test('create dan read', () async {
    await service.simpan(buatCatatan('1', 'Widget'));
    expect(service.jumlah, 1);
    expect(service.ambil('1')?.judul, 'Widget');
  });

  test('update menimpa data dengan id yang sama', () async {
    final c = buatCatatan('1', 'Widget');
    await service.simpan(c);
    await service.simpan(c.copyWith(judul: 'Widget Dasar'));
    expect(service.jumlah, 1);
    expect(service.ambil('1')?.judul, 'Widget Dasar');
  });

  test('delete dan hapusSemua', () async {
    await service.simpan(buatCatatan('1', 'A'));
    await service.simpan(buatCatatan('2', 'B'));
    await service.hapus('1');
    expect(service.ambil('1'), isNull);
    expect(service.jumlah, 1);
    await service.hapusSemua();
    expect(service.ambilSemua(), isEmpty);
  });

  test('data tetap ada setelah box ditutup dan dibuka lagi', () async {
    await service.simpan(buatCatatan('1', 'Persisten'));
    await Hive.close(); // seperti aplikasi ditutup
    final serviceBaru = await CatatanService.buka();
    final c = serviceBaru.ambil('1');
    expect(c?.judul, 'Persisten');
    expect(c?.dibuatPada, DateTime(2025, 1, 1, 8, 0)); // adapter bekerja
  });
}
```

```
flutter test
```

```
00:00 +0: loading test/catatan_service_test.dart
00:00 +0: create dan read
00:00 +1: update menimpa data dengan id yang sama
00:00 +2: delete dan hapusSemua
00:00 +3: data tetap ada setelah box ditutup dan dibuka lagi
00:00 +4: All tests passed!
```

Test terakhir paling penting: ia menutup Hive (seperti aplikasi ditutup), membuka box lagi, lalu memastikan data dan tanggalnya terbaca dengan benar. Itu bukti adapter kita bekerja dua arah. Dengan cara yang sama kamu juga bisa menguji `CatatanController`: buat `CatatanController(service)`, panggil `onInit()`, lalu periksa `daftar` setelah `tambah` atau `cari`, karena controller tidak membutuhkan widget sama sekali.

ℹ️ **Pengembangan lanjutan**

- Tambahkan field `disematkan` (pin) dengan menaikkan versi adapter menjadi 2 seperti contoh `DosenAdapter` di bagian 5.
- Ganti tombol hapus dengan gestur geser (`Dismissible`) dan SnackBar "Batalkan" yang menyimpan ulang catatan.
- Di Modul 14, pemisahan lapisan yang sama dipakai lagi (di sana disebut repository): data utama dari Supabase, lalu Hive menjadi cache offline.

11. Kesalahan Umum dan Debugging

Kumpulan error yang paling sering ditemui mahasiswa saat memakai Hive, lengkap dengan penyebab dan cara memperbaikinya.

Pesan error / gejala	Penyebab	Solusi
`HiveError: Box not found. Did you forget to call Hive.openBox()?`	`Hive.box('x')` dipanggil sebelum `openBox('x')`, atau nama box salah ketik.	Buka semua box di `main()` s Simpan nama box dalam konst (`CatatanService.namaBox`
`HiveError: Cannot write, unknown type: Catatan. Did you forget to register an adapter?`	Adapter belum didaftarkan.	Panggil `Hive.registerAdapter(Ca` setelah `initFlutter` dan se
`HiveError: There is already a TypeAdapter for typeId 0.`	Dua adapter memakai typeId sama, atau adapter yang sama didaftarkan dua kali (sering terjadi di test).	Beri typeId berbeda untuk seti cek dulu dengan `if (!Hive.isAdapterRegister`
`Cannot read, unknown typeId: 34. Did you forget to register an adapter?`	File box berisi objek bertipe yang adapternya belum didaftarkan, atau typeId-nya pernah diubah.	Daftarkan adapter yang benar. typeId setelah ada data. Saat c uninstall aplikasi untuk mengh
`RangeError` / data acak saat membaca objek	Urutan `read()` tidak sama dengan `write()`, atau field ditambah tanpa migrasi.	Samakan urutan. Pakai byte ve adapter hasil generate (bagian
`type '_Map<dynamic, dynamic>' is not a subtype of type 'Map<String, dynamic>'`	Map yang dibaca ulang dari file selalu bertipe `Map<dynamic, dynamic>`.	`Map<String, dynamic>.fr` buat `fromMap(Map map)`.
"Binding has not yet been initialized"	`await Hive.initFlutter()` dipanggil sebelum `WidgetsFlutterBinding.ensureInitialized()`.	Jadikan `ensureInitialized` di `main()`.

Pesan error / gejala	Penyebab	Solusi
UI tidak berubah setelah data disimpan	Menulis ke box tanpa memperbarui nilai `.obs` di controller (atau tanpa `setState`), atau tidak memakai `listenable()`.	Lewatkan perubahan lewat co menyimpan ke service lalu me (dan bungkus UI dengan `Obx` dengan `ValueListenableBu`
Data hilang setelah aplikasi ditutup paksa	`put` tidak di-`await`, dan aplikasi mati sebelum data tertulis ke file.	Selalu `await` operasi tulis ya
Data "hilang" setelah ganti nama box atau ganti package	Nama box baru berarti file baru. Box lama masih ada tetapi tidak dibuka.	Jangan mengganti nama box t datanya.

Hot reload vs hot restart

- **Hot reload** tidak menjalankan ulang `main()`. Jika kamu baru menambah `openBox` atau `registerAdapter` di `main()`, perubahannya belum terpakai, sehingga error "Box not found" bisa muncul. Lakukan **hot restart** (tombol ⟳ atau tekan R di terminal).
- **Hot restart** menjalankan ulang `main()` dari awal, tetapi file box tetap ada di perangkat. Data **tidak** ter-reset. Untuk benar-benar mengosongkan data, uninstall aplikasi atau panggil `box.clear()`.
- Setelah mengubah adapter dengan cara yang tidak kompatibel (mengubah urutan field), hot restart akan mencoba membaca data lama dengan adapter baru dan bisa crash. Uninstall aplikasi saat masih tahap development.

Tips debugging

```
// 1. Lihat seluruh isi box di Debug Console
debugPrint(Hive.box('pengaturan').toMap().toString());

// 2. Cek apakah box sudah terbuka sebelum dipakai
if (!Hive.isBoxOpen('catatan_box')) {
  await Hive.openBox<Catatan>('catatan_box');
}

// 3. Hapus satu box beserta filenya (misalnya tombol "Reset data" saat development)
await Hive.deleteBoxFromDisk('catatan_box');

// 4. Tangkap HiveError agar aplikasi tidak langsung crash
try {
  await Hive.openBox<Catatan>('catatan_box');
} on HiveError catch (e) {
```

```
  debugPrint('Gagal membuka box: ${e.message}');
}
```

Ringkasan modul

- Box = kumpulan key-value yang tersimpan permanen. Buka sekali di `main()`, lalu baca secara sinkron.
- `put` berfungsi untuk create sekaligus update. Pakai key unik (id), hindari index.
- Map dan List bisa langsung disimpan; ingat `Map.from` saat membaca.
- Objek sendiri butuh TypeAdapter dengan `typeId` unik yang tidak boleh berubah; tulis manual atau generate dengan `@GenerateAdapters`.
- UI reaktif lewat `box.listenable()`, atau pola MVC: service untuk Hive, `GetxController` dengan `.obs`, dan view dengan `Obx`.
- Data sensitif: `HiveAesCipher` dengan kunci di `flutter_secure_storage`.

Tugas Praktikum: Praktikum 11: Aplikasi Catatan Kuliah Offline dengan Hive

Sasaran Praktikum

Membangun aplikasi CRUD offline yang menyimpan objek buatan sendiri di Hive dengan arsitektur MVC GetX (model, service, controller, view), lalu mengembangkannya dengan fitur tambahan.

Langkah Pengerjaan

1. **Tugas 1 (Wajib, 20 poin):** Bangun ulang mini project Catatan Kuliah Offline dari bagian 10 sampai berjalan di emulator. Buktikan data tetap ada setelah aplikasi ditutup paksa dan dibuka kembali (lampirkan 2 screenshot).
2. **Tugas 2 (Wajib, 20 poin):** Tambahkan field `bool disematkan` (pin) pada model `Catatan`. Naikkan versi `CatatanAdapter` menjadi 2 dan baca field baru hanya jika versi >= 2, sehingga catatan lama dari Tugas 1 tetap terbaca tanpa uninstall.
3. **Tugas 3 (Wajib, 15 poin):** Tampilkan catatan yang disematkan selalu di urutan teratas (apa pun pilihan urutannya) dengan ikon pin, dan sediakan tombol untuk menyematkan atau melepas sematan.
4. **Tugas 4 (Wajib, 15 poin):** Tambahkan filter mata kuliah berupa deretan `FilterChip` di bawah kolom pencarian. Daftar mata kuliah diambil otomatis dari catatan yang ada.
5. **Tugas 5 (Wajib, 15 poin):** Simpan pilihan urutan terakhir pengguna ke box `pengaturan` (gunakan service terpisah, misalnya `PengaturanService`, yang dipanggil dari `CatatanController`) sehingga urutan tidak kembali ke default saat aplikasi dibuka ulang.
6. **Tugas 6 (Wajib, 15 poin):** Tulis unit test untuk `CatatanService` memakai `Hive.init` di folder sementara, minimal mencakup tambah, edit, hapus, dan persistensi setelah `Hive.close()`. Semua test harus lulus dengan `flutter test`.
7. **Bonus (+10 poin):** Ganti tombol hapus dengan gestur geser `Dismissible` dan SnackBar berisi aksi "Batalkan" yang mengembalikan catatan yang terhapus.

Output yang Diharapkan

Aplikasi Catatan Kuliah yang berjalan sepenuhnya offline: daftar catatan dengan pin, pencarian, filter mata kuliah, dan urutan yang diingat; form tambah/edit dengan validasi; konfirmasi hapus; data tetap ada setelah restart. Disertai output flutter test yang semua lulus dan flutter analyze tanpa issue.

Rubrik Penilaian

Tugas 1: fungsionalitas CRUD dan bukti persistensi (20%). Tugas 2: migrasi adapter tanpa kehilangan data lama (20%). Tugas 3: logika dan tampilan pin (15%). Tugas 4: filter mata kuliah (15%). Tugas 5: pengaturan urutan tersimpan dengan service terpisah (15%). Tugas 6: kualitas dan kelulusan unit test (15%). Bonus Dismissible + Batalkan (+10%). Pengurangan nilai jika view/widget mengakses Hive secara langsung, controller mengimpor material.dart, atau flutter analyze masih menampilkan issue.

Kuis Pemahaman

1. Kenapa modul ini memakai package hive_ce dan hive_ce_flutter, bukan hive?

A. Package hive 2.x sudah tidak dirawat, sedangkan hive_ce adalah lanjutan versi komunitas yang API-nya hampir sama dan terus diperbarui

B. hive_ce adalah database SQL, sedangkan hive adalah NoSQL

C. hive tidak bisa dipakai di Android

D. hive_ce membutuhkan koneksi internet agar lebih cepat

2. Apa urutan yang benar di dalam main() sebelum runApp?

A. openBox, lalu Hive.initFlutter, lalu registerAdapter

B. WidgetsFlutterBinding.ensureInitialized, await Hive.initFlutter, Hive.registerAdapter, await Hive.openBox

C. registerAdapter, lalu runApp, lalu Hive.initFlutter di dalam build()

D. Hive.box, lalu Hive.openBox, lalu ensureInitialized

3. Sebuah box sudah berisi data dengan key 'nama' = 'Budi'. Apa yang terjadi saat kode await box.put('nama', 'Siti') dijalankan?

A. Muncul error karena key sudah ada

B. Box sekarang berisi dua data dengan key 'nama'

C. Nilai lama ditimpa sehingga box.get('nama') menghasilkan 'Siti'

D. Data baru disimpan dengan key angka otomatis

4. Sebuah Map disimpan ke box lalu aplikasi di-restart. Kode final m = box.get('profil') as Map<String, dynamic>; tiba-tiba error. Apa penyebab dan solusinya?

A. Hive tidak bisa menyimpan Map, jadi harus memakai TypeAdapter

B. Map yang dibaca ulang dari file bertipe Map<dynamic, dynamic>, sehingga harus disalin dengan Map<String, dynamic>.from(...)

C. Box harus dibuka dengan enkripsi agar Map terbaca

D. Key Map harus berupa int

5. Manakah aturan typeId yang BENAR pada TypeAdapter?

A. Boleh sama untuk semua model asalkan nama kelasnya berbeda

B. Sebaiknya diganti setiap kali aplikasi di-update

C. Harus unik untuk setiap tipe dan tidak boleh diubah setelah ada data yang tersimpan

D. Harus sama dengan jumlah field pada model

6. Pada adapter manual, kamu menambah field baru 'email' dan langsung memanggil readString() keempat di read(). Aplikasi lama yang sudah berisi data menjadi crash. Cara aman untuk mencegahnya adalah...

A. Menyisipkan field email di posisi pertama

B. Menulis byte versi format sejak awal, menambah field baru di akhir, dan membacanya hanya jika versi data mendukung

C. Mengganti typeId setiap kali menambah field

D. Menghapus await pada box.put

7. Apa fungsi box.listenable() yang dipasangkan dengan ValueListenableBuilder?

A. Mengenkripsi box secara otomatis

B. Membuka box secara async di dalam build()

C. Memberi kabar setiap isi box berubah sehingga builder dijalankan ulang dan UI ikut berubah tanpa setState

D. Mengirim data box ke server

8. Pada pola MVC dengan GetX di modul ini, pengguna menyimpan catatan baru tetapi daftar di layar tidak berubah. Kode controller: await _service.simpan(catatan); (tanpa baris lain). Apa perbaikan yang tepat?

A. Memanggil Hive.box('catatan_box') langsung di view agar data terbaru terbaca

B. Setelah menyimpan, memperbarui state reaktif controller (misalnya _semua.assignAll(_service.ambilSemua())) sehingga Obx yang membacanya dibangun ulang

C. Mengganti Obx dengan StatelessWidget biasa

D. Membuka ulang box di dalam build()

9. Di mana kunci enkripsi untuk HiveAesCipher sebaiknya disimpan?

A. Ditulis langsung sebagai konstanta String di kode Dart

B. Disimpan di box Hive biasa bernama 'kunci'

C. Disimpan di flutter_secure_storage (Keystore/Keychain), dibuat sekali dengan Hive.generateSecureKey()

D. Dibuat ulang secara acak setiap aplikasi dibuka

Kunci Jawaban & Pembahasan

1. A — hive_ce (Community Edition) melanjutkan pengembangan hive yang sudah ditinggalkan pembuatnya. API inti seperti openBox, put, get, dan TypeAdapter tetap sama, sehingga yang berbeda hanya nama package dan import.

2. B — Binding harus siap sebelum memakai plugin, Hive harus tahu folder penyimpanannya, adapter harus terdaftar sebelum box yang memakai tipe tersebut dibuka, dan box dibuka sebelum dipakai di UI.

3. C — put() berfungsi sebagai create sekaligus update. Key yang sama akan menimpa value lama. Key angka otomatis hanya dibuat oleh add().

4. B — Setelah dibaca dari disk, Hive mengembalikan Map<dynamic, dynamic>. Cast langsung ke Map<String, dynamic> gagal; gunakan Map<String, dynamic>.from() atau fromMap yang menerima Map.

5. C — Hive menulis typeId di depan setiap objek untuk mencari adapter saat membaca. typeId ganda memicu HiveError, dan mengganti typeId membuat data lama tidak bisa dibaca.

6. B — Adapter manual membaca data berurutan tanpa nama field. Dengan byte versi, read() tahu apakah data lama memiliki field baru. Adapter hasil generate menyelesaikan masalah ini dengan menulis indeks setiap field.

7. C — listenable() dari hive_ce_flutter menghasilkan ValueListenable yang memicu rebuild ValueListenableBuilder setiap ada put, add, delete, atau clear.

8. B — Obx hanya mendengarkan nilai .obs yang dibacanya, bukan isi box. Box memang sudah berubah, tetapi controller harus memperbarui RxList/nilai .obs miliknya agar view ikut diperbarui. View sebaiknya tidak menyentuh Hive langsung.

9. C — Kunci di kode atau di box biasa mudah dibaca orang lain. Kunci yang dibuat ulang setiap kali membuat data lama tidak bisa didekripsi. Secure storage menyimpan kunci di penyimpanan aman milik sistem operasi.